

```

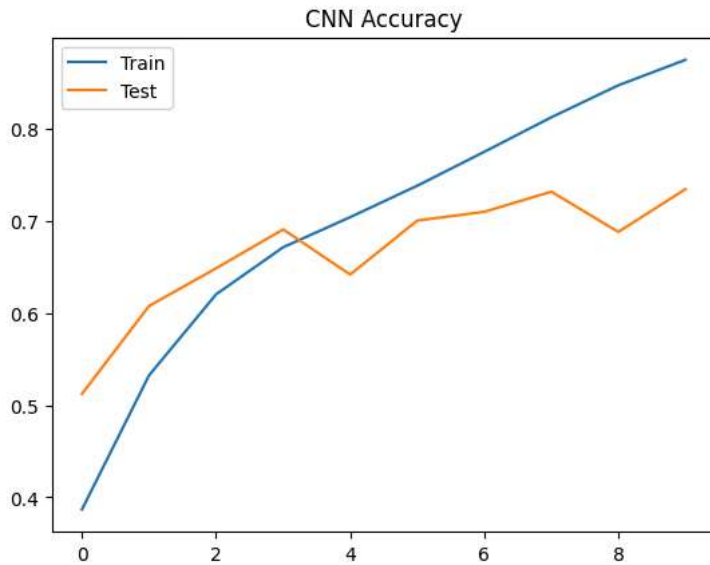
1 import tensorflow as tf
2 from tensorflow import keras
3 from tensorflow.keras import layers
4 import tensorflow_datasets as tfds
5 import numpy as np
6 import matplotlib.pyplot as plt
7 from sklearn.metrics import classification_report, confusion_matrix
8 from sklearn.preprocessing import MinMaxScaler
9 import pandas as pd
10
11 # Load dataset
12 dataset, info = tfds.load('tf_flowers', split='train', as_supervised=True, with_info=True)
13 images, labels = [], []
14 for img, label in tfds.as_numpy(dataset):
15     img = tf.image.resize(img, (128,128))
16     images.append(img.numpy())
17     labels.append(label)
18
19 images = np.array(images) / 255.0
20 labels = np.array(labels)
21
22 # Split 80:20
23 split = int(0.8 * len(images))
24 x_train, x_test = images[:split], images[split:]
25 y_train, y_test = labels[:split], labels[split:]
26
27 # Build CNN model
28 cnn_model = keras.Sequential([
29     layers.Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)),
30     layers.MaxPooling2D(2,2),
31
32     layers.Conv2D(64, (3,3), activation='relu'),
33     layers.MaxPooling2D(2,2),
34
35     layers.Conv2D(128, (3,3), activation='relu'),
36     layers.MaxPooling2D(2,2),
37
38     layers.Flatten(),
39     layers.Dense(128, activation='relu'),
40     layers.Dropout(0.5),
41     layers.Dense(5, activation='softmax')
42 ])
43
44 cnn_model.compile(optimizer='adam',
45                  loss='sparse_categorical_crossentropy',
46                  metrics=['accuracy'])
47
48 # Train CNN
49 history = cnn_model.fit(
50     x_train, y_train,
51     epochs=10,
52     validation_data=(x_test, y_test)
53 )
54
55 # Evaluate
56 test_loss, test_acc = cnn_model.evaluate(x_test, y_test)
57 print("CNN Test Accuracy:", test_acc)
58
59 # Plot accuracy
60 plt.plot(history.history['accuracy'], label='Train')
61 plt.plot(history.history['val_accuracy'], label='Test')
62 plt.title("CNN Accuracy")
63 plt.legend()
64 plt.show()
65

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape` / `input_
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
92/92 ━━━━━━━━━━━ 11s 80ms/step - accuracy: 0.3869 - loss: 1.4052 - val_accuracy: 0.5123 - val_loss: 1.1953
Epoch 2/10
92/92 ━━━━━━━━━━━ 2s 23ms/step - accuracy: 0.5324 - loss: 1.1356 - val_accuracy: 0.6076 - val_loss: 0.9253
Epoch 3/10
92/92 ━━━━━━━━━━━ 2s 22ms/step - accuracy: 0.6202 - loss: 0.9898 - val_accuracy: 0.6485 - val_loss: 0.9145
Epoch 4/10
92/92 ━━━━━━━━━━━ 2s 22ms/step - accuracy: 0.6713 - loss: 0.8715 - val_accuracy: 0.6907 - val_loss: 0.8446
Epoch 5/10
92/92 ━━━━━━━━━━━ 2s 24ms/step - accuracy: 0.7040 - loss: 0.7644 - val_accuracy: 0.6417 - val_loss: 0.9254
Epoch 6/10
92/92 ━━━━━━━━━━━ 2s 24ms/step - accuracy: 0.7381 - loss: 0.6835 - val_accuracy: 0.7003 - val_loss: 0.7903
Epoch 7/10
92/92 ━━━━━━━━━━━ 2s 23ms/step - accuracy: 0.7749 - loss: 0.6061 - val_accuracy: 0.7098 - val_loss: 0.7371
Epoch 8/10
92/92 ━━━━━━━━━━━ 3s 25ms/step - accuracy: 0.8123 - loss: 0.4947 - val_accuracy: 0.7316 - val_loss: 0.7589
Epoch 9/10
92/92 ━━━━━━━━━━━ 2s 23ms/step - accuracy: 0.8471 - loss: 0.4014 - val_accuracy: 0.6880 - val_loss: 1.0225
Epoch 10/10
92/92 ━━━━━━━━━━━ 2s 23ms/step - accuracy: 0.8747 - loss: 0.3259 - val_accuracy: 0.7343 - val_loss: 0.8349
23/23 ━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.7343 - loss: 0.8349
CNN Test Accuracy: 0.7343324422836304

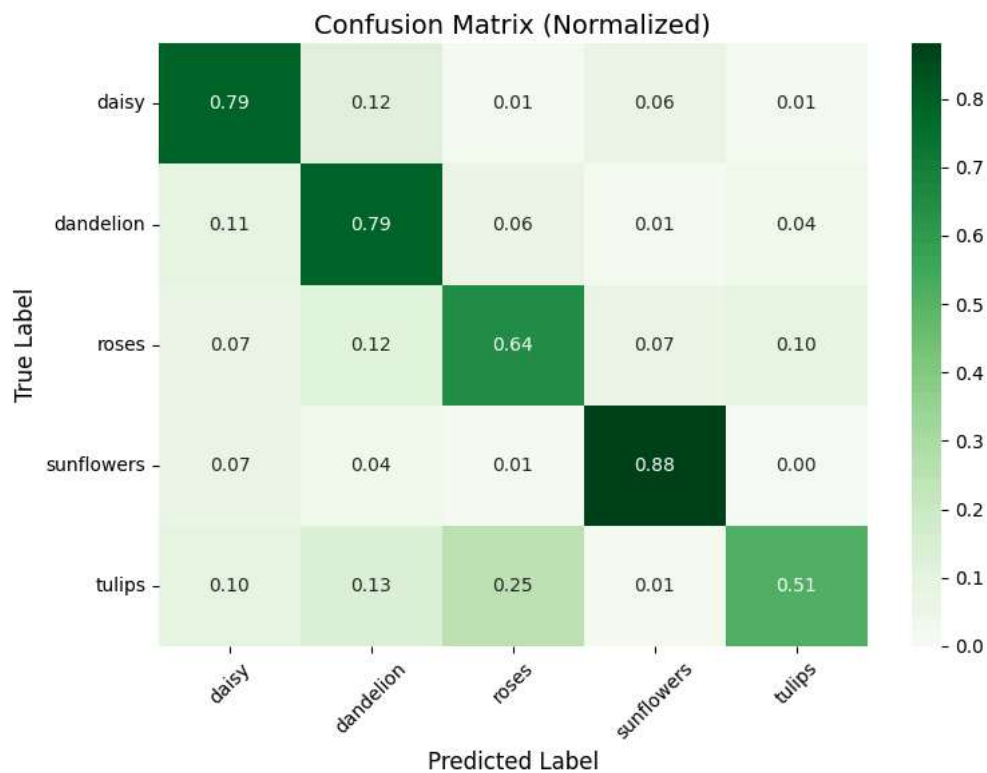
```



```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import numpy as np
4 from sklearn.metrics import confusion_matrix
5
6 # Generate confusion matrix
7 cm = confusion_matrix(y_test, y_pred)
8
9 # Normalize
10 cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
11
12 class_names = ['daisy', 'dandelion', 'roses', 'sunflowers', 'tulips']
13
14 plt.figure(figsize=(8,6))
15
16 sns.heatmap(
17     cm_normalized,
18     annot=True,
19     fmt=".2f",
20     cmap="Greens",
21     xticklabels=class_names,
22     yticklabels=class_names,
23     cbar=True
24 )
25
26 plt.title("Confusion Matrix (Normalized)", fontsize=14)
27 plt.xlabel("Predicted Label", fontsize=12)
28 plt.ylabel("True Label", fontsize=12)
29
30 plt.xticks(rotation=45)
31 plt.yticks(rotation=0)
32
33 plt.tight_layout()
34 plt.show()

```



```

1 import tensorflow as tf
2 from tensorflow import keras
3 from tensorflow.keras import layers
4 import tensorflow_datasets as tfds
5 import matplotlib.pyplot as plt
6 import numpy as np
7 from sklearn.metrics import classification_report, confusion_matrix
8 from sklearn.preprocessing import MinMaxScaler
9 import pandas as pd
10
11 AUTOTUNE = tf.data.AUTOTUNE
12 from google.colab import files
13 uploaded = files.upload()
14
15 filename = list(uploaded.keys())[0]
16 df = pd.read_csv(filename)
17
18 print("Columns in dataset:", df.columns)
19
20 # CHANGE COLUMN NAME HERE IF NEEDED
21 data = df['Close'].values.reshape(-1,1)
22
23 # Normalize
24 scaler = MinMaxScaler()
25 data = scaler.fit_transform(data)
26
27 # Create sequences
28 X, y = [], []
29 for i in range(60, len(data)):
30     X.append(data[i-60:i])
31     y.append(data[i])
32
33 X, y = np.array(X), np.array(y)
34
35 # Split 80:20
36 split = int(0.8 * len(X))
37 X_train, X_test = X[:split], X[split:]
38 y_train, y_test = y[:split], y[split:]
39
40 # Build RNN model
41 rnn_model = keras.Sequential([
42     layers.LSTM(50, return_sequences=True, input_shape=(60,1)),
43     layers.LSTM(50),
44     layers.Dense(25),
45     layers.Dense(1)
46 ])
47
48 rnn_model.compile(optimizer='adam', loss='mse')
49
50 # Train RNN
51 rnn_model.fit(
52     X_train, y_train,
53     epochs=20,
54     batch_size=32,
55     validation_data=(X_test, y_test)
56 )

```

```

57 )
58
59 # Predict
60 predictions = rnn_model.predict(X_test)
61
62 # Inverse transform
63 predictions = scaler.inverse_transform(predictions)
64 y_test_actual = scaler.inverse_transform(y_test)
65
66 # Plot results
67 plt.plot(y_test_actual, label='Actual')
68 plt.plot(predictions, label='Predicted')
69 plt.title("Stock Price Prediction")
70 plt.legend()
71 plt.show()

```

Choose files A6_efb80bf...6a5fe9df.csv

A6_efb80bf9-7531-4cb5-bd45-6eea6a5fe9df.csv(text/csv) - 36996 bytes, last modified: 14/04/2026 - 100% done

Saving A6_efb80bf9-7531-4cb5-bd45-6eea6a5fe9df.csv to A6_efb80bf9-7531-4cb5-bd45-6eea6a5fe9df (1).csv

Columns in dataset: Index(['Close'], dtype='object')

Epoch 1/20

/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to
super().__init__(**kwargs)

49/49 ————— 3s 22ms/step - loss: 0.0086 - val_loss: 0.0018

Epoch 2/20

49/49 ————— 1s 14ms/step - loss: 3.5752e-04 - val_loss: 0.0016

Epoch 3/20

49/49 ————— 1s 14ms/step - loss: 3.0792e-04 - val_loss: 0.0018

Epoch 4/20

49/49 ————— 1s 14ms/step - loss: 2.9766e-04 - val_loss: 0.0015

Epoch 5/20

49/49 ————— 1s 14ms/step - loss: 3.1893e-04 - val_loss: 0.0016

Epoch 6/20

49/49 ————— 1s 15ms/step - loss: 3.2663e-04 - val_loss: 0.0014

Epoch 7/20

49/49 ————— 1s 13ms/step - loss: 2.6204e-04 - val_loss: 0.0013

Epoch 8/20

49/49 ————— 1s 21ms/step - loss: 2.5348e-04 - val_loss: 0.0013

Epoch 9/20

49/49 ————— 1s 21ms/step - loss: 2.3510e-04 - val_loss: 0.0012

Epoch 10/20

49/49 ————— 1s 19ms/step - loss: 2.3862e-04 - val_loss: 0.0011

Epoch 11/20

49/49 ————— 1s 14ms/step - loss: 2.5897e-04 - val_loss: 0.0025

Epoch 12/20

49/49 ————— 1s 15ms/step - loss: 2.3933e-04 - val_loss: 0.0013

Epoch 13/20

49/49 ————— 1s 15ms/step - loss: 2.1852e-04 - val_loss: 0.0015

Epoch 14/20

49/49 ————— 1s 14ms/step - loss: 2.4012e-04 - val_loss: 0.0018

Epoch 15/20

49/49 ————— 1s 14ms/step - loss: 2.3056e-04 - val_loss: 9.4118e-04

Epoch 16/20

49/49 ————— 1s 14ms/step - loss: 1.8051e-04 - val_loss: 9.6955e-04

Epoch 17/20

49/49 ————— 1s 14ms/step - loss: 1.8919e-04 - val_loss: 0.0021

Epoch 18/20

49/49 ————— 1s 15ms/step - loss: 2.0900e-04 - val_loss: 9.6928e-04

Epoch 19/20

49/49 ————— 1s 14ms/step - loss: 1.6990e-04 - val_loss: 7.9794e-04

Epoch 20/20

49/49 ————— 1s 13ms/step - loss: 1.8231e-04 - val_loss: 9.5859e-04

13/13 ————— 0s 20ms/step

Stock Price Prediction

